

Introduction

Programmazione Avanzata 2025-26

Note

- Starting from the academic year 2025-26, the course “**Programmazione Avanzata**” replaces, at 2nd year, the courses
 - “**Programmazione a oggetti**” (previously taught using Java, 3rd year)
 - “**Tecniche di programmazione**” (previously taught in Python, 3rd year)
- The course will build upon the fundamentals of programming in Python and the algorithmic aspects covered in the **Informatica** course of the 1st year, introducing the object-oriented paradigm and exploring advanced programming topics related, among others, to the use of complex data structures, database querying, and the development of applications with interactive user interfaces

Goals

- The course aims to extend the student's programming skills, enabling them to tackle and **solve real-world problems**, including complex ones, through the development of **advanced software applications**
- Specifically, the course presents the **object-oriented programming** paradigm, analyzing its advantages in software production and **applying it to concrete scenarios** using the Python language

Goals

- It then integrates the skills acquired in previous courses, particularly Informatica and Basi di Dati, with the goal of providing the tools to create applications capable of presenting **interactive user interfaces**, **accessing relational databases**, and graphically displaying processing results.
- Finally, it delves into **algorithmic aspects** by introducing the concept of recursive programming, analyzing common data structures (especially graphs), and studying some search, optimization, and simulation algorithms with **practical applications** in logistics, production, and business organization

Learning outcomes

- Knowledge and understanding:
 - Fundamental concepts of object-oriented programming (classes, objects, attributes, methods, inheritance, polymorphism) and complex data structures (lists, trees, graphs, queues) in Python
 - Code management (and version control) and collaborative programming
 - Tools for developing user interfaces and accessing relational databases from Python applications
 - Main programming patterns; recursion; search and optimization problems (divide-and-conquer techniques)

Learning outcomes

- Applying knowledge and understanding:
 - Proficiency in object-oriented programming in Python and in the use of platforms for collaborative code management
 - Ability to analyze the requirements of advanced management-type software problems in order to define an appropriate object-oriented design
 - Skills in developing Python applications with interactive user interfaces and access to relational databases
 - Ability to address search, optimization, and simulation problems through the definition and implementation of algorithmic solutions

Pre-requisites

- **Basic programming concepts** (data types and structures, control statements, subprograms and subprogram calls)
- Ability to **solve simple problems by defining algorithmic solutions** and **writing software programs in the Python**
 - Starting from this basic knowledge of Python, students will be presented the object-orientation paradigm and how to apply it using Python, then moving to more advanced topics
- **Knowledge of relational databases** and, more in particular, their **querying through SQL statements**
 - Databases will be created using graphics tools, then accessed through Python using (rather) simple queries

Syllabus

- 4 CFU – Object-oriented programming paradigm. Advanced aspects of programming in Python: classes and objects, collections, database access, main programming patterns (factory, MVC and user interfaces, DAO and access to relational databases). Platforms for code management and collaborative programming.
- 2 CFU – Advanced data structures: graphs, trees, queues and related Python libraries.
- 2 CFU – Algorithms: recursion, optimization, graph traversal and search procedures, shortest paths, simulation.
- 2 CFU – Problem-solving exercises: search and optimization problems; exhaustive (recursive) search techniques; applications to management-related cases (event simulation, scheduling problems, resource optimization, transport search, etc.).

Topics per week (tentative plan)

- | | |
|-----------------------------|--------------------|
| 1. Python (procedural), Git | 8. Recursion |
| 2. OOP | 9. Graphs |
| 3. OOP and Python | 10. Graphs |
| 4. Advanced data structures | 11. Graphs |
| 5. GUI (Flet) | 12. Complexity |
| 6. Database access (MySQL) | 13. Exam exercises |
| 7. Recursion | 14. Exam exercises |

Organization

- **Classroom lectures** (60h) and **exercises** (20h), for each student
 - No formal distinction between lesson and exercise hours: theoretical aspects and their practical application are addressed in an integrated way
 - Students are encouraged to attend these hours with their own laptop and carry out activities (e.g, live coding) in parallel with the instructor
- **Lab sessions** (20h) to let students work, with teaching staff support, on problems related to topics tackled in class
 - Problem published beforehand on GitHub, solution published later, and possibly presented during classroom hours
 - Correct completion of the exercises leads to max 2 additional points
- Additional, optional hours (lab assistant/s only)
 - Defined based on students and lab assistants' availability, online

Calendar

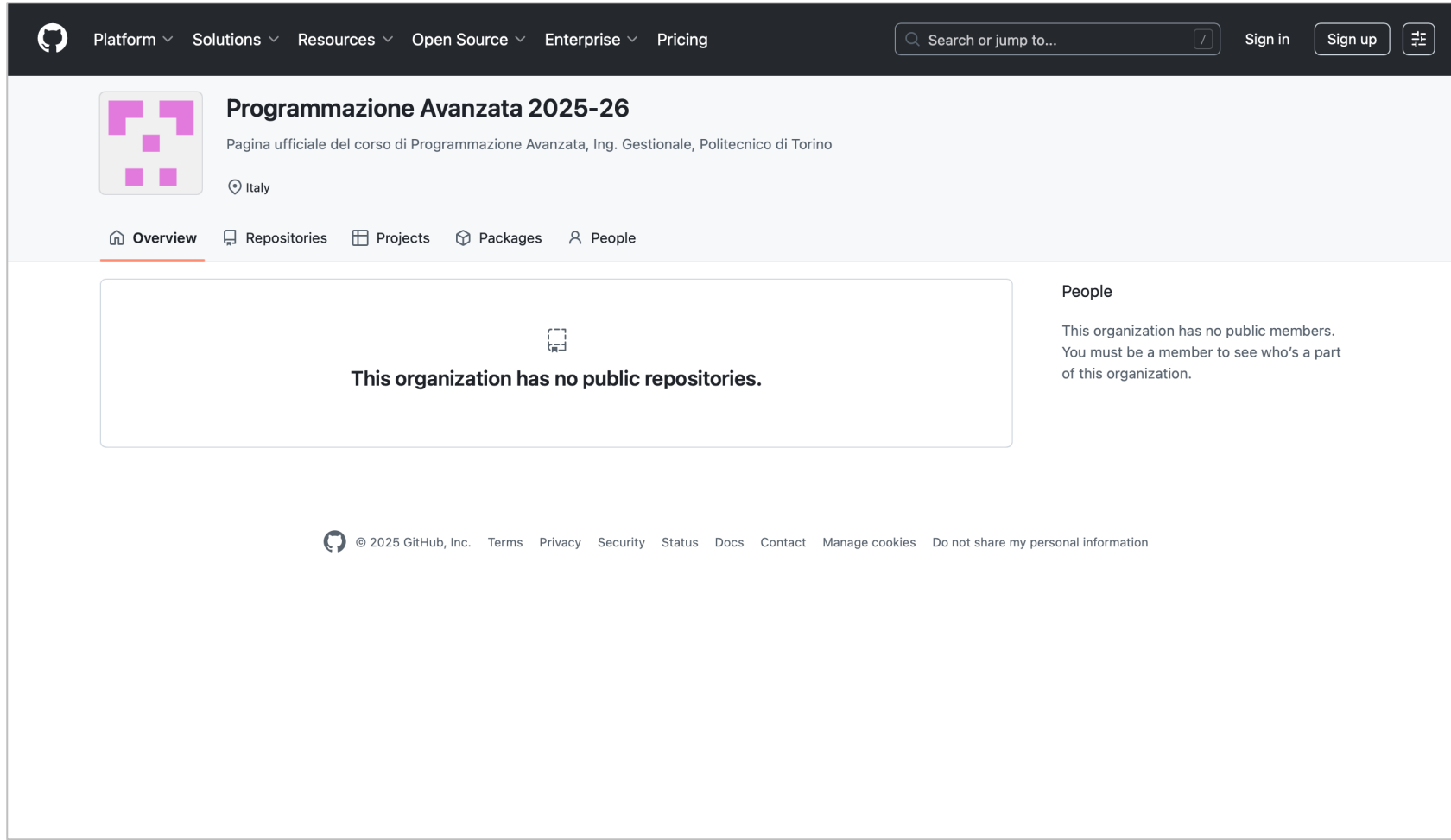
	Monday	Tuesday	Wednesday	Thursday	Friday
8:30-10:00					
10:00-11:30	Room 29				
11:30-13:00					
13:00-14:30		Room 5T (LAB-G2)*	LAIB1T (LAB-G1)*	Room 29	
14:30-16:00					
16:00-17:30					
17:30-19:00					

* Division by lastname (G1 o G2) published on the Portale della Didattica. Each week the two groups will alternate. Students are requested to agree on any group changes with the instructors.

Exercises and code on GitHub

- To share code (lab assignments with base Python projects, exercises developed in class, solutions of lab problems, solutions developed by students, etc.), the GitHub platform will be used
 - <https://github.com/Programmazione-Avanzata-2025-26/>
- How to signup on GitHub (to be done as soon as possible)
 - Create an account at <https://github.com/signup>
 - Choose a username
 - If the institutional email is used (e.g., s123456@studenti.polito.it), student discounts are unlocked
 - However, the suggestion is to choose an account which remains relevant also after the studies

Exercises and code on GitHub



Tools

- Python
 - In the lab, v3.11.5
- PyCharm Community Edition
 - In the lab, v2025.2.1.1
- MariaDB or MySQL
- Dbeaver
- Open source libraries

Teaching staff

- Fabrizio Lamberti
 - Dip. Automatica e Informatica, 3rd floor
 - fabrizio.lamberti@polito.it
 - Consultation dates to be agreed upon by email
- Alessandro Visconti
 - Dip. Automatica e Informatica, 2nd floor (LAB2)
 - alessandro.visconti@polito.it
- Roberta Macaluso
 - Dip. Automatica e Informatica, 2nd floor (LAB2)
 - roberta.macaluso@polito.it
- Lab assistants
 - Alessandro Cocco (353655), Riccardo Vittimberga (344810)

Teaching material

- Reference website: Portale della Didattica, with
 - Slides (in English) & recordings
 - In-class exercises (problems+Python solutions) ...
 - ... added topic by topic, on GitHub
 - Lab exercises
- Books
 - Concetti di informatica e fondamenti di Python, Cay S. Horstmann, Rance D. Necaise, Apogeo
 - Programmazione in Python, Kenneth A. Lambert, Apogeo
 - Algoritmi e strutture dati: Guida pragmatica con esempi in Python, Jay Wengrow, Apogeo
 - Data Science con Python, Dmitry Zinoviev, Apogeo

Learning approach

- Attending classroom lessons and exercises
- Try to solve lab problems autonomously, asking for help when needed
- Try to work in parallel with the instructors on own laptop
- Seek for additional help using, e.g., Python official documentation, the debugger, search engines (Google), question-and-answer platforms for programmers like Stack Overflow, ...

Final exam procedure

- The exam, with a duration of about **2 hours**, takes place in the lab and consists of writing a program in Python and structured according to the object-oriented paradigm, split in several parts, capable of solving a management-related problem while meeting the given requirements, **starting from a base project** provided by the instructors
- The assigned problem will have a level of complexity similar to that of the laboratory exercises assigned in the last weeks
- Based on a provided database, the goal will be to design the program with appropriate data structures (collections and graphs) and to implement the functionalities to query the database, perform data processing (using recursive programming techniques, simulation, and search and optimization algorithms), and present the results

Final exam assessment

- The grade, expressed on a scale from 0 to 30, will be assigned according to the quality of the submitted program, measured in terms of appropriateness and correctness of the implementation
- During the exam, students may use all materials provided or developed during the course, as well as textbooks and notes in either paper or digital format.
- Honors (cum laude) will be awarded upon achieving a score equal to or greater than 31.50, considering
 - The 30 points obtainable from the exam
 - Plus the 2 points achievable through lab exercises (submitted by the communicated dealines), which will be assessed together with the exam program